Dinâmica: Show do Milhão - Docker

Regras Gerais do Jogo:

1. **Estrutura das Perguntas:**
   O jogo é composto por **4 níveis de dificuldade**: Fácil, Médio, Difícil e o Nível Milhão.Serão lidas **4 perguntas por nível** (totalizando 16 perguntas no jogo).
   O apresentador lerá uma pergunta por vez.

2. **Dinâmica de Resposta:**
   Assim que o apresentador terminar de ler a pergunta, a **primeira equipe que levantar a mão** ganha o direito de responder.

3. **Pontuação e Penalidades:**
   Acerto: A equipe ganha a pontuação equivalente ao nível da pergunta.
   Erro: A equipe perde a pontuação equivalente ao nível da pergunta.
   A pontuação é acumulativa ao longo das rodadas.

4. **Nível Milhão:**
   As 4 últimas perguntas pertencem ao **Nível Milhão**.
   A dinâmica de resposta continua a mesma (quem levantar a mão primeiro responde).

   Acerto: A equipe ganha **1.000.000 de pontos**.
   Erro: A equipe perde **1.000.000 de pontos**.

5. **Vencedor:**
   Ao final da 16ª pergunta, a equipe que acumular a **maior quantidade de pontos** será a ganhadora.

Tabela de Pontuação

Nível	Perguntas	Acerto	Erro
Fácil	1 a 4	+10.000	-5.000

Médio	5 a 8	+50.000	-25.000
Difícil	9 a 12	+100.000	-50.000
Milhão	13 a 16	+1.000.000	-500.000

Perguntas - Nível fácil

1. **Qual é a finalidade principal do ambiente de homologação?**

   A) Desenvolver o código pela primeira vez
   B) Validar a aplicação antes de disponibilizá-la aos usuários
   C) Substituir o desenvolvimento
   D) Sincronizar os dados

2. **Durante o desenvolvimento de uma aplicação, um aluno precisa construir uma imagem Docker e iniciar um container. Qual alternativa descreve corretamente cada componente?**

   a) O Docker Hub constrói imagens localmente, o Dockerfile inicia containers e o Engine armazena imagens online.
   b) O Dockerfile define as instruções da imagem, o Engine gerencia as operações do Docker e o Hub permite compartilhar e baixar imagens.
   c) O Docker Engine contém o código-fonte, o Hub executa containers e o Dockerfile gerencia os processos em execução.
   d) O Dockerfile gerencia containers, o Engine funciona como catálogo de imagens e o Hub define as instruções de construção.

3. **Que arquivo impede que o .env com as senhas seja copiado para dentro da imagem durante o build?**

   a) .gitignore
   b) compose.yaml
   c) .dockerignore
   d) Dockerfile

4. **Qual sequência representa corretamente a criação e execução de uma aplicação com Docker?**

a) Aplicação → container → Dockerfile → imagem → `docker build`
b) Dockerfile → `docker run` → imagem → `docker build` → container
c) Aplicação → Dockerfile → `docker build` → imagem → `docker run` → container
d) Aplicação → imagem → Dockerfile → container → `docker run`

Perguntas - Nível médio

1. **Um sistema utiliza MySQL no desenvolvimento e SQLite no ambiente de homologação. Qual afirmação é mais correta?**

   A) Toda consulta MySQL falhará no SQLite
   B) Os dois bancos são exatamente iguais
   C) A aplicação sempre funcionará, pois ambos usam SQL
   D) Algumas consultas ou funcionalidades podem ser incompatíveis entre os bancos

2. **Uma equipe utiliza as imagens `python:3.12-slim` e `nginx:alpine`. Sobre as imagens Docker, qual alternativa está correta?**

   a) As palavras `slim` e `alpine` identificam containers em execução, enquanto `3.12` e `nginx` indicam as camadas utilizadas durante a execução.
   b) Cada tag representa uma camada independente da imagem, e imagens com tags diferentes não podem reaproveitar camadas em comum.
   c) As tags identificam versões ou variantes das imagens, enquanto as camadas compõem a imagem e podem ser reaproveitadas entre imagens compatíveis.
   d) As tags identificam exclusivamente a linguagem de programação utilizada, enquanto as camadas armazenam somente os arquivos temporários gerados pelo container.

3. **Se a imagem é a mesma em todos os ambientes, como a aplicação sabe qual banco usar e qual nível de log aplicar?**

   a) Existe uma versão do código para cada ambiente
   b) Pelas variáveis de ambiente definidas em cada ambiente
   c) O Docker detecta sozinho em que servidor está rodando
   d) A configuração fica gravada dentro da imagem no build

4. **No comando abaixo, qual é a função de `-p 8080:80`?**

   docker run -d -p 8080:80 app_aula:1.0

   a) Conectar a porta 8080 do container à porta 80 do PostgreSQL.
   b) Encaminhar a porta 8080 do computador para a porta 80 do container.

c) Executar 80 containers usando a porta 8080.
d) Alterar a porta interna do Nginx de 80 para 8080.

Perguntas - Nível difícil

1. **Uma equipe padronizou as dependências com Docker, mas a aplicação ainda falha porque a senha do banco está incorreta. O que isso demonstra?**

   A) Docker não tem utilidade
   B) Docker ajuda a padronizar o ambiente, mas não corrige automaticamente todas as configurações externas
   C) Senhas não influenciam conexões
   D) O código precisa ser todo reescrito

2. **Uma empresa deseja executar várias aplicações isoladas em um mesmo servidor, reduzindo o consumo de recursos em comparação com máquinas virtuais tradicionais. Qual alternativa explica melhor como os containers Docker podem ajudar nesse cenário?**

   a) Cada container executa um sistema operacional convidado completo, mas dispensa o uso de recursos do sistema operacional da máquina hospedeira.
   b) Os containers compartilham o kernel do sistema operacional hospedeiro e mantêm isolamento entre aplicações e seus ambientes, geralmente evitando a sobrecarga de um sistema operacional completo por container.
   c) O Docker Engine cria um kernel independente para cada container, garantindo que nenhuma biblioteca ou dependência seja compartilhada entre aplicações.
   d) Os containers eliminam a necessidade de isolamento entre processos, utilizando diretamente todas as bibliotecas instaladas no sistema operacional hospedeiro.

3. **No desenvolvimento, o código chega ao container por bind mount ou Compose Watch. E em homologação e produção?**

   a) Também por bind mount, apontando para a pasta do servidor
   b) O código é baixado do GitHub toda vez que o container sobe
   c) O Compose Watch sincroniza o código do desenvolvedor com o servidor
   d) O código vai dentro da própria imagem

4. **Uma embarcação foi cadastrada no PostgreSQL usando um volume nomeado. Depois, foram executados:**

   docker compose down
   docker compose up -d

O que deve acontecer com o cadastro?

a) Será apagado, pois down sempre remove containers e volumes.
b) Continuará salvo, pois o volume nomeado é preservado e reutilizado.
c) Será copiado para a imagem da aplicação durante o novo up.
d) Continuará salvo apenas se o container antigo possuir o mesmo ID.

Perguntas - Nível milhão

1. **Qual afirmação conecta corretamente a história do Docker aos problemas discutidos na apresentação?**

   A) O Docker surgiu em 2013, contribuiu para tornar ambientes mais consistentes, mas não inventou os containers nem eliminou todos os problemas.
   B) O Docker foi criado em 2014 e criou a tecnologia de containers para garantir consistência.
   C) O Docker criou os ambientes de desenvolvimento, homologação e produção.
   D) O Docker garante que uma aplicação sempre funcione em qualquer sistema operacional sem necessidade de configurações.

2. **Uma empresa utiliza Docker Swarm para gerenciar containers em várias máquinas, controlar réplicas e realizar atualizações. Qual alternativa descreve corretamente o funcionamento do Swarm?**

   a) O manager constrói e armazena imagens, enquanto os workers definem e distribuem as réplicas.
   b) O manager coordena o cluster e os serviços, enquanto os workers executam tarefas. O Swarm permite gerenciar réplicas e atualizações.
   c) Cada worker gerencia suas próprias réplicas, enquanto o manager apenas monitora os nós, sem gerenciar serviços.
   d) O manager coordena os serviços, mas cada container precisa de um sistema operacional completo, como nas máquinas virtuais.

3. **A versão app:1.4.0 foi para produção e apareceu um erro grave. Qual é a atitude correta?**

   a) Entrar no container de produção e corrigir o código direto nele
   b) Trocar a tag para latest para pegar a versão mais nova
   c) Rodar docker build no servidor de produção com a correção
   d) Fazer um rollback, subindo a imagem anterior para a produção continuar funcionando normalmente

4. **Duas pessoas executam o mesmo `compose.yaml` em computadores diferentes. Elas querem compartilhar os dados e visualizar alterações sem atualizar manualmente a página. Qual solução está tecnicamente correta?**

a) Criar volumes com o mesmo nome nos dois computadores, pois nomes iguais sincronizam os dados.
b) Compartilhar somente a imagem da aplicação, pois imagens também armazenam os novos cadastros.
d) Usar uma aplicação e um PostgreSQL centrais, fazendo todos acessarem o mesmo servidor, e implementar consultas periódicas, Server-Sent Events ou WebSocket para atualização automática das telas.
d) Publicar diretamente o PostgreSQL para todos os usuários, eliminando a aplicação e o controle de acesso.